Advanced Artificial Intelligence and Machine Learning Lab 7

October 20, 2026

1 Lab 7

> Adapted from Stanford CS336 *Language Modeling from Scratch*, Spring 2025, Assignment 1 "Building a Transformer LM" (`stanford-cs336/assignment1-basics`, tag `spring2025`), Copyright 2025 Stanford University, MIT license. Exercises 1-4 follow the handout's §3 problem by problem, checked against the assignment's own snapshot tests; Exercises 5-6 follow §4.1, §5.1, §6 and §7.3, scaled down for a CPU: raw bytes instead of a trained BPE tokenizer (§2 is skipped), PyTorch's AdamW instead of a hand-written one (§4.3), and a 0.8M-parameter model trained for 600 steps on the assignment's 5 MB TinyStories sample.

Session 9 opened with CS336's "simple, modern variant" of the 2017 Transformer: pre-norm RMSNorm, SwiGLU, RoPE and no biases. You build exactly that model, train it on children's stories, generate text - and take RoPE out again. As in the handout, the model code uses no `torch.nn`, `torch.nn.functional` or `torch.optim` beyond `nn.Parameter`, containers and `nn.init`.

> **Instructor timing (2h session).** ~10 min: intro and setup. ~25 min: Exercise 1 - `Linear`, RMSNorm, SwiGLU. ~15 min: Exercise 2 - RoPE. ~20 min: Exercise 3 - softmax and causal multi-head self-attention. ~15 min: Exercise 4 - the full LM and its accounting. ~30 min: Exercise 5 - loss, data loader, training (a ~4.5 minute run - discuss the questions while it runs) and decoding. Exercise 6 (another ~4.5 minute run) is an extension for fast groups or for afterwards.

1.1 Setup

These labs are designed to run on Google Colab - no local Python install needed. Open this notebook via Colab's GitHub loader (`colab.research.google.com/github/KarikS/lecture_advanced_ai_ml/blob/main/<path-to-this-notebook`) or File > Open notebook > GitHub tab, repo `KarikS/lecture_advanced_ai_ml`, then run the cell below once per session. Running the notebook locally instead (e.g. via the `exercises/python/` venv described in that folder's README) works unchanged - the cell below is a no-op there. No GPU is needed - Exercise 5's training run took about 4.5 minutes with 4 CPU threads in testing, and can take noticeably longer on Colab's free tier.

[1]:
```
import sys

IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
```

```
    import os
    if not os.path.exists('lecture_advanced_ai_ml'):
        !git clone https://github.com/KarikS/lecture_advanced_ai_ml.git
```

1.2 Imports

[2]:
```
import math
import time
import urllib.request
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F
from matplotlib_inline.backend_inline import set_matplotlib_formats
from torch import nn, Tensor

set_matplotlib_formats('png', 'pdf')
torch.manual_seed(0)
```

[2]:
```
<torch._C.Generator at 0x77dbc4f1d4b0>
```

1.3 Test Files from CS336

CS336 checks every component against *snapshots*: the outputs of the course staff's reference implementation for fixed inputs and fixed weights. We download the snapshots, the weights they were computed with (a small trained Transformer LM, 5.6 MB) and the TinyStories sample for Exercise 5 from the `spring2025` commit into `.data/cs336/` (`weights_only=True`: an untrusted `.pt` file is a pickle and could otherwise execute code).

[3]:
```
CS336_COMMIT = '430e2c844e29f8aad8f9330e8706db9cb508241f'  # tag `spring2025`
CS336_URL = ('https://raw.githubusercontent.com/stanford-cs336/
 ↪assignment1-basics/'
             f'{CS336_COMMIT}/tests/')
CS336_DIR = Path('.data/cs336')

SNAPSHOT_NAMES = [
    'test_linear', 'test_embedding', 'test_rmsnorm', 'test_swiglu', 'test_rope',
    'test_scaled_dot_product_attention', 'test_4d_scaled_dot_product_attention',
    'test_multihead_self_attention', 'test_multihead_self_attention_with_rope',
    'test_transformer_block', 'test_transformer_lm',␣
 ↪'test_transformer_lm_truncated_input',
]
FILES = (['fixtures/ts_tests/model.pt', 'fixtures/tinystories_sample_5M.txt']
         + [f'_snapshots/{name}.npz' for name in SNAPSHOT_NAMES])
```

```
for relative_path in FILES:
    target = CS336_DIR / relative_path
    if not target.exists():
        target.parent.mkdir(parents=True, exist_ok=True)
        urllib.request.urlretrieve(CS336_URL + relative_path, target)

reference_weights = torch.load(CS336_DIR / 'fixtures/ts_tests/model.pt',
                               map_location='cpu', weights_only=True)
# The reference model was trained with torch.compile, which prefixes every key.
reference_weights = {k.replace('_orig_mod.', ''): v for k, v in␣
 ↪reference_weights.items()}
print(list(reference_weights)[:10])
```

```
['token_embeddings.weight', 'layers.0.attn.q_proj.weight',
'layers.0.attn.k_proj.weight', 'layers.0.attn.v_proj.weight',
'layers.0.attn.output_proj.weight', 'layers.0.ffn.w1.weight',
'layers.0.ffn.w2.weight', 'layers.0.ffn.w3.weight', 'layers.0.ln1.weight',
'layers.0.ln2.weight']
```

The reference model has 3 layers, `d_model = 64`, 4 heads, `d_ff = 128` and 10,000 tokens. Its weight names are the attribute names your modules need for `load_state_dict`. The test inputs and `check`'s tolerances are the same as in CS336's `tests/conftest.py`.

[4]:
```
BATCH_SIZE, N_QUERIES, N_KEYS = 4, 12, 16
D_MODEL, N_HEADS, D_FF, ROPE_THETA = 64, 4, 128, 10000.0
VOCAB_SIZE, N_LAYERS = 10_000, 3

torch.manual_seed(1)
test_q = torch.randn(BATCH_SIZE, N_QUERIES, D_MODEL)
torch.manual_seed(2)
test_k = torch.randn(BATCH_SIZE, N_KEYS, D_MODEL)
torch.manual_seed(3)
test_v = torch.randn(BATCH_SIZE, N_KEYS, D_MODEL)
torch.manual_seed(4)
test_x = torch.randn(BATCH_SIZE, N_QUERIES, D_MODEL)  # "in_embeddings" in CS336
torch.manual_seed(5)
test_mask = torch.randn(BATCH_SIZE, N_QUERIES, N_KEYS) > 0.5
torch.manual_seed(6)
test_token_ids = torch.randint(0, VOCAB_SIZE, (BATCH_SIZE, N_QUERIES))


def check(test_name: str, actual: Tensor, atol: float = 1e-2, rtol: float =␣
 ↪1e-4) -> None:
    expected = np.load(CS336_DIR / '_snapshots' / f'{test_name}.npz')['array']
    np.testing.assert_allclose(actual.detach().numpy(), expected, atol=atol,␣
 ↪rtol=rtol,
                               err_msg=f'{test_name} does not match the␣
 ↪snapshot')
```

```
    print(f'{test_name}: passed')
```

1.4 Exercise 1: The Building Blocks (`Linear`, `Embedding`, RMSNorm, SwiGLU)

Handout §3.4-3.5.2. A `Linear` layer computes $y = Wx$ with **no bias term**, following most modern LLMs (Session 9, "More generally: dropping bias terms"). Store W with shape `(out_features, in_features)`, as `nn.Linear` does - applied to a batch of row vectors, that is `x @ W.T`. Initialize it with $\mathcal{N}\left(0, \frac{2}{d_{\text{in}}+d_{\text{out}}}\right)$ truncated at $[-3\sigma, 3\sigma]$, using `torch.nn.init.trunc_normal_` (its arguments `a` and `b` are absolute bounds).

[5]:
```
class Linear(nn.Module):
    def __init__(self, in_features: int, out_features: int, device=None,␣
 ↪dtype=None):
        super().__init__()
# TODO: Create self.weight as an nn.Parameter of shape (out_features,␣
 ↪in_features)
# and initialize it with nn.init.trunc_normal_ (mean 0, std as above, truncated
# at -3 std and +3 std).

    def forward(self, x: Tensor) -> Tensor:
# TODO: Apply the linear map to the last dimension of x.


# Given: an embedding is just indexing into a (vocab_size, d_model) matrix.
class Embedding(nn.Module):
    def __init__(self, num_embeddings: int, embedding_dim: int, device=None,␣
 ↪dtype=None):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(num_embeddings, embedding_dim,␣
 ↪device=device, dtype=dtype))
        nn.init.trunc_normal_(self.weight, mean=0.0, std=1.0, a=-3.0, b=3.0)

    def forward(self, token_ids: Tensor) -> Tensor:
        return self.weight[token_ids]
```

[6]:
```
linear = Linear(D_MODEL, D_FF)
linear.load_state_dict({'weight': reference_weights['layers.0.ffn.w1.weight']})
check('test_linear', linear(test_x))

embedding = Embedding(VOCAB_SIZE, D_MODEL)
embedding.load_state_dict({'weight': reference_weights['token_embeddings.
 ↪weight']})
check('test_embedding', embedding(test_token_ids))
```

RMSNorm (Session 9, "LayerNorm vs RMSNorm") does not subtract the mean or add a bias, as LayerNorm does - it only rescales:

$$\mathrm{RMSNorm}(a_i) = \frac{a_i}{\mathrm{RMS}(a)}\, g_i, \qquad \mathrm{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$$

with a learned gain g and $\varepsilon = 10^{-5}$, computed in `float32`.

[7]:
```
class RMSNorm(nn.Module):
    def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model, device=device,␣
 ↪dtype=dtype))

    def forward(self, x: Tensor) -> Tensor:
        in_dtype = x.dtype
        x = x.to(torch.float32)
# TODO: Compute RMS(x) over the last dimension (keepdim=True, so it broadcasts),
# divide x by it and multiply by the gain self.weight. Call the result `result`.
        return result.to(in_dtype)
```

[8]:
```
rmsnorm = RMSNorm(D_MODEL)
rmsnorm.load_state_dict({'weight': reference_weights['layers.1.ln1.weight']})
check('test_rmsnorm', rmsnorm(test_x), atol=1e-6)
```

The SwiGLU feed-forward network (used e.g. by Llama 3 and Qwen 2.5) replaces $W_2\,\mathrm{ReLU}(W_1 x)$ by a *gated* version: SiLU, $x \cdot \sigma(x)$, multiplied element-wise with a second projection, the gate:

$$\mathrm{FFN}(x) = W_2\left(\mathrm{SiLU}(W_1 x) \odot W_3 x\right), \qquad W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}},\ W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$$

[9]:
```
def silu(x: Tensor) -> Tensor:
    return x * torch.sigmoid(x)


class SwiGLU(nn.Module):
    def __init__(self, d_model: int, d_ff: int):
        super().__init__()
        self.w1 = Linear(d_model, d_ff)
        self.w2 = Linear(d_ff, d_model)
        self.w3 = Linear(d_model, d_ff)

    def forward(self, x: Tensor) -> Tensor:
```

```
# TODO: Implement W2 (SiLU(W1 x) * W3 x).
```

[10]:
```
swiglu = SwiGLU(D_MODEL, D_FF)
swiglu.load_state_dict({f'{w}.weight': reference_weights[f'layers.0.ffn.{w}.
 ↪weight']
                        for w in ['w1', 'w2', 'w3']})
check('test_swiglu', swiglu(test_x), atol=1e-5)
```

Question: Without gating, nearly every Transformer sets $d_{\text{ff}} = 4\,d_{\text{model}}$. With a GLU variant, most models instead use $d_{\text{ff}} \approx \frac{8}{3}\,d_{\text{model}}$ (Session 9, "Exception #1 - GLU variants"). Where does $\frac{8}{3}$ come from? Count the feed-forward parameters in both cases.

1.5 Exercise 2: Rotary Position Embeddings (RoPE)

Handout §3.5.3. RoPE injects position by *rotating* queries and keys. For a vector $q^{(i)} \in \mathbb{R}^d$ at position i, rotate each consecutive pair (q_{2k-1}, q_{2k}), $k = 1, \ldots, d/2$, by the angle $\theta_{i,k} = i/\Theta^{(2k-2)/d}$:

$$\begin{pmatrix} q'_{2k-1} \\ q'_{2k} \end{pmatrix} = \begin{pmatrix} \cos\theta_{i,k} & -\sin\theta_{i,k} \\ \sin\theta_{i,k} & \cos\theta_{i,k} \end{pmatrix} \begin{pmatrix} q_{2k-1} \\ q_{2k} \end{pmatrix}, \qquad \Theta = 10000.$$

Keys are rotated the same way. Precompute cos and sin of all angles up to `max_seq_len` once, as buffers (they are not trained). In 0-based indexing, the pairs are `x[..., 0::2]` and `x[..., 1::2]`, and pair k has angle $i/\Theta^{2k/d}$.

[11]:
```
class RotaryPositionalEmbedding(nn.Module):
    def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None):
        super().__init__()
# TODO: Compute all angles, shape (max_seq_len, d_k // 2), and register their cos
# and sin with self.register_buffer('cos', ..., persistent=False) (same for␣
 ↪'sin').

    def forward(self, x: Tensor, token_positions: Tensor) -> Tensor:
# TODO: Index cos and sin with token_positions, rotate every (even, odd) pair of␣
 ↪x
# and write the results into a new tensor (torch.empty_like(x)).
```

[12]:
```
rope = RotaryPositionalEmbedding(ROPE_THETA, d_k=D_MODEL, max_seq_len=N_QUERIES)
check('test_rope', rope(test_x, torch.arange(N_QUERIES)), atol=1e-6)
```

Session 9 called RoPE a *relative* position embedding, although every vector is rotated by its *absolute* position. We place the same query at position i and the same key at position j, rotate both, and compute their dot product - the attention score:

[13]:
```
rope_demo = RotaryPositionalEmbedding(ROPE_THETA, d_k=16, max_seq_len=128)
torch.manual_seed(0)
q_vec, k_vec = torch.randn(16), torch.randn(16)


def rotated_score(i: int, j: int) -> float:
    q_rot = rope_demo(q_vec[None], torch.tensor([i]))[0]
    k_rot = rope_demo(k_vec[None], torch.tensor([j]))[0]
    return float(q_rot @ k_rot)


for i, j in [(5, 3), (12, 10), (100, 98), (5, 5), (60, 60), (3, 5), (40, 42)]:
    print(f'query at {i:3d}, key at {j:3d} (offset {i - j:+d}): score␣
 ↪{rotated_score(i, j):+.4f}')
```

Question: What do you observe, and why does it follow from the math? Why is this property useful for a language model?

1.6 Exercise 3: Softmax and Causal Multi-Head Self-Attention

Handout §3.5.4-3.5.5. $\exp(v_i)$ overflows for large v_i, but softmax doesn't change if the same constant is subtracted from every entry - so subtract the maximum first. Then scaled dot-product attention, as in Session 7: $\text{softmax}\left(QK^\top/\sqrt{d_k}\right)V$. The optional boolean `mask` has shape `(..., queries, keys)`, and `True` **means the query may attend to that key** (masked-out scores become $-\infty$). Both must work for any number of leading batch dimensions.

[14]:
```
def softmax(x: Tensor, dim: int) -> Tensor:
# TODO: Subtract the maximum along `dim` (keepdim=True), exponentiate, normalize.


def scaled_dot_product_attention(Q: Tensor, K: Tensor, V: Tensor, mask: Tensor |␣
 ↪None = None) -> Tensor:
# TODO: Compute Q K^T / sqrt(d_k) (transpose the last two dimensions of K), fill␣
 ↪the
# positions where mask is False with -inf (Tensor.masked_fill), apply your␣
 ↪softmax
# over the keys, and multiply with V.
```

[15]:
```
# CS336's test_softmax_matches_pytorch, including the overflow case (+100).
softmax_input = torch.tensor([[0.4655, 0.8303, 0.9608, 0.9656, 0.6840],
                              [0.2583, 0.2198, 0.9334, 0.2995, 0.1722],
                              [0.1573, 0.6860, 0.1327, 0.7284, 0.6811]])
assert torch.allclose(softmax(softmax_input, dim=-1), F.softmax(softmax_input,␣
 ↪dim=-1), atol=1e-6)
```

```
assert torch.allclose(softmax(softmax_input + 100, dim=-1), F.
 ↪softmax(softmax_input, dim=-1), atol=1e-6)
print('softmax: passed')

check('test_scaled_dot_product_attention',
      scaled_dot_product_attention(test_q, test_k, test_v, test_mask), atol=1e-6)
# The same inputs with an extra "head" dimension: (batch, head, seq, d).
q4, k4, v4 = (t.reshape(BATCH_SIZE // 2, 2, *t.shape[1:]) for t in (test_q,␣
 ↪test_k, test_v))
mask4 = test_mask.reshape(BATCH_SIZE // 2, 2, N_QUERIES, N_KEYS)
check('test_4d_scaled_dot_product_attention', scaled_dot_product_attention(q4,␣
 ↪k4, v4, mask4), atol=1e-6)
```

Causal multi-head self-attention, with h heads and $d_k = d_v = d_{\text{model}}/h$:

$$\text{MultiHeadSelfAttention}(x) = W_O \,\text{Concat}(\text{head}_1, \ldots, \text{head}_h), \qquad \text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

Q, K, V take **one** projection each for all heads; reshape `(..., seq, d_model)` to `(..., seq, heads, d_k)` and move the heads in front of the sequence, as a batch dimension.

- **Causal mask:** token i may only attend to positions $j \le i$ (`torch.tril`).
- **RoPE** is applied to queries and keys (not values), after splitting into heads, with the same rotation for every head. If `token_positions` is not given, use `0, ..., seq_len - 1`.

[16]:
```
class CausalMultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, rope:␣
 ↪RotaryPositionalEmbedding | None = None):
        super().__init__()
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        self.q_proj = Linear(d_model, d_model)
        self.k_proj = Linear(d_model, d_model)
        self.v_proj = Linear(d_model, d_model)
        self.output_proj = Linear(d_model, d_model)
        self.rope = rope

    def forward(self, x: Tensor, token_positions: Tensor | None = None) ->␣
 ↪Tensor:
        *batch_dims, seq_len, d_model = x.shape
# TODO:
# 1. Project x to q, k, v and reshape each to (*batch_dims, num_heads, seq_len,␣
 ↪d_k).
# 2. If self.rope is set, rotate q and k.
# 3. Build the (seq_len, seq_len) causal mask and call␣
 ↪scaled_dot_product_attention.
```

```
# 4. Merge the heads back into d_model and apply output_proj.
```

[17]:
```
attn_weights = {f'{p}_proj.weight': reference_weights[f'layers.0.attn.{p}_proj.
 ↪weight']
                for p in ['q', 'k', 'v', 'output']}

mhsa = CausalMultiHeadSelfAttention(D_MODEL, N_HEADS)
mhsa.load_state_dict(attn_weights)
check('test_multihead_self_attention', mhsa(test_x), atol=1e-6)

mhsa_rope = CausalMultiHeadSelfAttention(
    D_MODEL, N_HEADS, RotaryPositionalEmbedding(ROPE_THETA, D_MODEL // N_HEADS,␣
 ↪N_KEYS))
mhsa_rope.load_state_dict(attn_weights)
check('test_multihead_self_attention_with_rope',
      mhsa_rope(test_x, torch.arange(N_QUERIES)[None]), atol=1e-6)

# Leaking future tokens is the classic silent bug of a language model, so we␣
 ↪test the causal
# property directly: changing the last input must not change any earlier output.
x_changed = test_x.clone()
x_changed[:, -1] += 10.0
with torch.no_grad():
    out_original, out_changed = mhsa_rope(test_x), mhsa_rope(x_changed)
print('max change at positions 0 .. n-2:', float((out_original[:, :-1] -␣
 ↪out_changed[:, :-1]).abs().max()))
print('max change at the last position: ', float((out_original[:, -1] -␣
 ↪out_changed[:, -1]).abs().max()))
```

Question: Why can the causal model process a training sequence of length m in a single forward pass and still get m separate next-token predictions, each of which only sees its own prefix?

1.7 Exercise 4: The Full Transformer LM

Handout §3.6. A pre-norm block normalizes the *input* of each sub-layer and adds the output to the residual stream, which itself is never normalized:

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)), \qquad y = z + \text{SwiGLU}(\text{RMSNorm}(z))$$

The LM embeds the tokens, runs `num_layers` blocks, applies a final RMSNorm (because the residual stream is never normalized) and a linear LM head that returns one *logit* per vocabulary entry. All layers share one RoPE module.

[18]:
```
class TransformerBlock(nn.Module):
    def __init__(self, d_model: int, num_heads: int, d_ff: int,
                 rope: RotaryPositionalEmbedding | None):
        super().__init__()
        self.ln1 = RMSNorm(d_model)
        self.attn = CausalMultiHeadSelfAttention(d_model, num_heads, rope)
        self.ln2 = RMSNorm(d_model)
        self.ffn = SwiGLU(d_model, d_ff)

    def forward(self, x: Tensor) -> Tensor:
# TODO: The two pre-norm sub-layers, each with its residual connection.


class TransformerLM(nn.Module):
    def __init__(self, vocab_size: int, context_length: int, d_model: int,␣
 ↪num_layers: int,
                 num_heads: int, d_ff: int, rope_theta: float):
        super().__init__()
        self.context_length = context_length
        self.token_embeddings = Embedding(vocab_size, d_model)
        rope = RotaryPositionalEmbedding(rope_theta, d_model // num_heads,␣
 ↪context_length)
        self.layers = nn.ModuleList(
            [TransformerBlock(d_model, num_heads, d_ff, rope) for _ in␣
 ↪range(num_layers)])
        self.ln_final = RMSNorm(d_model)
        self.lm_head = Linear(d_model, vocab_size)

    def forward(self, token_ids: Tensor) -> Tensor:
# TODO: Embed, run all blocks, final norm, LM head.
```

[19]:
```
block = TransformerBlock(D_MODEL, N_HEADS, D_FF,
                         RotaryPositionalEmbedding(ROPE_THETA, D_MODEL //␣
 ↪N_HEADS, N_KEYS))
block.load_state_dict({k.replace('layers.0.', ''): v
                       for k, v in reference_weights.items() if k.
 ↪startswith('layers.0.')})
check('test_transformer_block', block(test_x), atol=1e-6)

reference_lm = TransformerLM(VOCAB_SIZE, N_KEYS, D_MODEL, N_LAYERS, N_HEADS,␣
 ↪D_FF, ROPE_THETA)
reference_lm.load_state_dict(reference_weights)
check('test_transformer_lm', reference_lm(test_token_ids), atol=1e-4, rtol=1e-2)
check('test_transformer_lm_truncated_input', reference_lm(test_token_ids[:, :
 ↪N_QUERIES // 2]), atol=1e-4)
```

That is the complete model, checked against CS336's reference with its trained weights.

Resource accounting (handout problem `transformer_accounting`): how big is this architecture at real scale, and where does the compute go? `forward_flops` counts only matrix multiplies (an $(m \times n) \cdot (n \times p)$ product costs $2mnp$ FLOPs).

[20]:
```
def count_parameters(vocab_size: int, num_layers: int, d_model: int, d_ff: int)␣
 ↪-> int:
# TODO: Count the parameters of TransformerLM: token embeddings, per layer␣
 ↪(attention
# projections, SwiGLU, two RMSNorm gains), final RMSNorm, LM head. (No biases;␣
 ↪RoPE
# has no parameters.)


def forward_flops(vocab_size: int, T: int, num_layers: int, d_model: int, d_ff:␣
 ↪int) -> dict[str, int]:
    return {
        'attention projections': num_layers * 4 * 2 * T * d_model * d_model,
        'attention scores + sum': num_layers * 2 * 2 * T * T * d_model,
        'feed-forward (SwiGLU)': num_layers * 3 * 2 * T * d_model * d_ff,
        'LM head': 2 * T * d_model * vocab_size,
    }


assert count_parameters(VOCAB_SIZE, N_LAYERS, D_MODEL, D_FF) == sum(p.numel()␣
 ↪for p in reference_lm.parameters())

# GPT-2 shapes with our architecture. The handout gives d_ff = 6400 for XL only;␣
 ↪for the
# others we use GPT-2's own d_ff = 4 * d_model.
gpt2_shapes = {  # (context_length, num_layers, d_model, d_ff)
    'GPT-2 small':           (1024, 12, 768, 4 * 768),
    'GPT-2 medium':          (1024, 24, 1024, 4 * 1024),
    'GPT-2 large':           (1024, 36, 1280, 4 * 1280),
    'GPT-2 XL':              (1024, 48, 1600, 6400),
    'GPT-2 XL, 16k context': (16384, 48, 1600, 6400),
}
for name, (T, L, d, d_ff) in gpt2_shapes.items():
    n_params = count_parameters(50_257, L, d, d_ff)
    flops = forward_flops(50_257, T, L, d, d_ff)
    total = sum(flops.values())
    shares = ', '.join(f'{k} {v / total:5.1%}' for k, v in flops.items())
    print(f'{name:22s} {n_params / 1e9:5.2f}B params ({4 * n_params / 1e9:5.2f}␣
 ↪GB in float32), '
          f'{total / 1e12:6.1f} TFLOPs: {shares}')
```

Question: Which part of the model needs the most FLOPs, and how do the shares shift from GPT-2 small to XL? What happens when GPT-2 XL's context grows from 1,024 to 16,384 tokens?

1.8 Exercise 5: Training and Generating Text

Handout §4.1, §5.1, §6, §7.2. The loss is the cross-entropy between the logits o_i and the true next token x_{i+1}, $\ell_i = \log \sum_a \exp(o_i[a]) - o_i[x_{i+1}]$, averaged over all positions; its exponential is the **perplexity**. As with softmax, subtract the maximum logit first.

[21]:
```
def cross_entropy(logits: Tensor, targets: Tensor) -> Tensor:
# TODO: logits (..., vocab_size), targets (...). Subtract the max logit, compute
# log-sum-exp minus the target's logit (Tensor.gather), return the mean.


# CS336's test_cross_entropy, including the overflow case (logits x 1000).
ce_logits = torch.tensor([[0.1088, 0.1060, 0.6683, 0.5131, 0.0645],
                          [0.4538, 0.6852, 0.2520, 0.3792, 0.2675],
                          [0.4578, 0.3357, 0.6384, 0.0481, 0.5612],
                          [0.9639, 0.8864, 0.1585, 0.3038, 0.0350],
                          [0.3356, 0.9013, 0.7052, 0.8294, 0.8334],
                          [0.6333, 0.4434, 0.1428, 0.5739, 0.3810],
                          [0.9476, 0.5917, 0.7037, 0.2987, 0.6208],
                          [0.8541, 0.1803, 0.2054, 0.4775, 0.8199]])
ce_targets = torch.tensor([1, 0, 2, 2, 4, 1, 4, 0])
for scale in [1.0, 1000.0]:
    assert torch.allclose(cross_entropy(scale * ce_logits, ce_targets),
                          F.cross_entropy(scale * ce_logits, ce_targets),␣
 ↪atol=1e-4)
print('cross_entropy: passed')
```

The data. The tokens are the text's UTF-8 **bytes** (`vocab_size = 256`), with every `<|endoftext|>` replaced by byte `0`. We split by position (first 90% train, last 10% validation): random windows would overlap, and validation would partly measure memorization.

The data loader (§5.1). For a start index s, the input is $x_s, \ldots, x_{s+m-1}$ and the target the same window shifted by one. Sample the start indices uniformly from all valid ones - the target window must still fit into the array.

[22]:
```
EOT_ID = 0

raw_text = (CS336_DIR / 'fixtures/tinystories_sample_5M.txt').
 ↪read_text(encoding='utf-8')
assert '\x00' not in raw_text
tokens = np.frombuffer(raw_text.replace('<|endoftext|>', '\x00').
 ↪encode('utf-8'), dtype=np.uint8)
tokens = tokens.astype(np.int64)
```

```
n_train_tokens = int(0.9 * len(tokens))
train_tokens, val_tokens = tokens[:n_train_tokens], tokens[n_train_tokens:]
print(f'{len(train_tokens):,} training tokens, {len(val_tokens):,} validation␣
 ↪tokens')

# Generated bytes may not be valid UTF-8, or may form characters that never␣
 ↪occur in the
# training text; both are shown as escapes such as \xe2. The backslash is␣
 ↪allowed explicitly,
# since the escapes themselves start with one.
TRAIN_CHARS = set(raw_text) | {'\\'}


def bytes_to_text(ids: list[int]) -> str:
    text = bytes(ids).decode('utf-8', errors='backslashreplace')
    return ''.join(ch if ch in TRAIN_CHARS else ch.encode('unicode_escape').
 ↪decode('ascii')
                   for ch in text)


def get_batch(dataset: np.ndarray, batch_size: int, context_length: int, device:␣
 ↪str = 'cpu',
              generator: torch.Generator | None = None) -> tuple[Tensor, Tensor]:
# TODO: Sample batch_size start indices (torch.randint with the given generator),
# build the input and target windows, each (batch_size, context_length), move␣
 ↪both
# to `device`.


# The deterministic parts of CS336's test_get_batch.
seen_starts = set()
for _ in range(1000):
    x, y = get_batch(np.arange(0, 100), batch_size=32, context_length=7)
    assert x.shape == y.shape == (32, 7) and torch.equal(x + 1, y)
    seen_starts.update(x[:, 0].tolist())
assert min(seen_starts) == 0 and max(seen_starts) == 100 - 7 - 1
print('get_batch: passed')
print(repr(bytes_to_text(train_tokens[:200].tolist())))
```

Training. The handout's recipe with smaller numbers: `d_model = 128` (handout: 512), 4 layers and 4 heads (handout: 4 and 16), `d_ff = 320` (the multiple of 64 closest to $\frac{8}{3} \cdot 128$), context 128 (handout: 256), batch 32, 600 steps (2.5M tokens instead of 327M), AdamW, gradient clipping (§4.5) and the cosine schedule (§4.4).

[23]:
```
LM_CONFIG = dict(vocab_size=256, context_length=128, d_model=128, num_layers=4,␣
↪num_heads=4,
                 d_ff=320, rope_theta=10000.0)
TRAIN_STEPS, BATCH, MAX_LR, WARMUP_STEPS, EVAL_EVERY = 600, 32, 1e-3, 30, 50

torch.set_num_threads(4)


def cosine_lr(step: int, max_lr: float, min_lr: float, warmup_steps: int,␣
↪total_steps: int) -> float:
    if step < warmup_steps:
        return max_lr * step / warmup_steps
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return min_lr + 0.5 * (1 + math.cos(math.pi * progress)) * (max_lr - min_lr)


val_batches = [get_batch(val_tokens, 64, LM_CONFIG['context_length'],
                         generator=torch.Generator().manual_seed(1000 + i)) for␣
↪i in range(8)]


def validation_loss(model: nn.Module) -> float:
    model.eval()
    with torch.no_grad():
        losses = [float(cross_entropy(model(x), y)) for x, y in val_batches]
    model.train()
    return sum(losses) / len(losses)


def train_lm(model: nn.Module, seed: int = 0, steps: int = TRAIN_STEPS, max_lr:␣
↪float = MAX_LR) -> list[tuple]:
    optimizer = torch.optim.AdamW(model.parameters(), lr=max_lr, betas=(0.9, 0.
↪95), weight_decay=0.1)
    batch_generator = torch.Generator().manual_seed(seed)
    history = [(0, validation_loss(model))]
    t0 = time.time()
    for step in range(1, steps + 1):
        for group in optimizer.param_groups:
            group['lr'] = cosine_lr(step, max_lr, 0.1 * max_lr, WARMUP_STEPS,␣
↪steps)
        x, y = get_batch(train_tokens, BATCH, LM_CONFIG['context_length'],␣
↪generator=batch_generator)
# TODO: One training step: forward pass, your cross_entropy, zero the gradients,
# backward pass, clip the gradient norm to 1.0 (torch.nn.utils.clip_grad_norm_),
# optimizer step. Call the loss `loss`.
        if step % EVAL_EVERY == 0:
```

```
            val_loss = validation_loss(model)
            history.append((step, val_loss))
            print('STEP:\t{:4}\tTRAIN LOSS:\t{:.3f}\tVAL LOSS:\t{:.3f}\tVAL␣
↪PERPLEXITY:\t{:.2f}\tELAPSED:\t{:.0f}s'
                  .format(step, float(loss), val_loss, math.exp(val_loss), time.
↪time() - t0))
    return history


torch.manual_seed(0)
lm = TransformerLM(**LM_CONFIG)
print(f'{sum(p.numel() for p in lm.parameters()):,} parameters')
lm_history = train_lm(lm)
```

The validation loss should fall from about $\log 256 \approx 5.5$ (a uniform guess over all bytes) to roughly 1.1-1.2 nats per byte - when this lab was tested over three seeds, it ended at 1.11-1.19 (a byte-level perplexity of about 3.0-3.3).

Question: The handout reports its loss in nats per *BPE token* (CPU target: 2.00). Why can't our nats per *byte* be compared to it directly - and which direction does the difference go?

Generating text (§6). Feed the model a prompt, take the logits at the *last* position, sample the next token, append it and repeat - until the end-of-text token or a maximum length. The model only ever sees the last 128 tokens. Two tricks from the handout (and Session 8): **temperature** τ samples from softmax(v/τ), and **top-p** keeps only the smallest set of most likely tokens whose probabilities sum to at least p: sort descending and keep a token if the mass *before* it is still below p (so the most likely token is always kept), then renormalize.

[24]:
```
def next_token_distribution(logits: Tensor, temperature: float, top_p: float) ->␣
↪Tensor:
# TODO: Apply the temperature and your softmax; then top-p: sort descending
# (torch.sort), compute the mass before each token (cumsum minus the token␣
↪itself),
# zero the tokens whose preceding mass is >= top_p, and renormalize. Return the
# probabilities in the original token order.


@torch.no_grad()
def generate(model: TransformerLM, prompt: str, max_new_tokens: int = 250,␣
↪temperature: float = 1.0,
             top_p: float = 1.0, seed: int = 0) -> str:
    model.eval()
    generator = torch.Generator().manual_seed(seed)
    ids = list(prompt.encode('utf-8'))
    for _ in range(max_new_tokens):
        context = torch.tensor([ids[-model.context_length:]])
# TODO: Logits of the last position -> next_token_distribution -> one sample with
```

```
# torch.multinomial(..., 1, generator=generator). Stop at EOT_ID, else append.
    return bytes_to_text(ids)


toy_logits = torch.tensor([3.0, 2.0, 1.0, 0.0])
assert torch.allclose(next_token_distribution(toy_logits, 1.0, 1.0), F.
 ↪softmax(toy_logits, -1))
print(next_token_distribution(toy_logits, temperature=1.0, top_p=0.7))  # keeps␣
 ↪the top 2

for temperature, top_p in [(1e-3, 1.0), (0.8, 0.9), (1.5, 1.0)]:
    print(f'\n--- temperature {temperature}, top-p {top_p} ---')
    print(generate(lm, 'Once upon a time', temperature=temperature, top_p=top_p))
```

Don't expect the handout's fluent stories (20 times the parameters, over 100 times the tokens): expect real words and TinyStories phrases, but little sense beyond a few words.

Question: What do the three settings produce, and why? The handout asks for "at least two factors" that affect how good the generated text is - which do you see here?

1.9 Exercise 6 (Extension): Taking RoPE Out Again

Handout §7.3, "Ablation 2". Without any position embedding (NoPE), a causal Transformer can in theory still infer positions - token i attends over exactly i tokens. How much does RoPE contribute? Remove it from every layer and train with the same data, seed and steps.

[25]:
```
torch.manual_seed(0)
lm_nope = TransformerLM(**LM_CONFIG)
# TODO: Remove the position embedding from every layer of lm_nope, then train it␣
 ↪with
# train_lm and store the result in nope_history.
```

[26]:
```
plt.plot(*zip(*lm_history), marker='o', label='RoPE')
plt.plot(*zip(*nope_history), marker='o', label='NoPE')
plt.axhline(math.log(256), color='gray', linestyle='--', label='uniform over 256␣
 ↪bytes')
plt.xlabel('step')
plt.ylabel('validation loss (nats per byte)')
plt.ylim(0, 6)
plt.legend()
plt.show()
print(generate(lm_nope, 'Once upon a time', temperature=0.8, top_p=0.9))
```

In testing over three seeds, NoPE ended at 1.85-1.90 nats per byte (RoPE: 1.11-1.19), and its

samples are mostly not even words.

Question: Why does NoPE struggle so much *here*? Look at what its generated text gets wrong.

1.10 Conclusion

Here's what you should take away from this lab:

- Session 9's "simple, modern variant" is only a handful of small changes to the 2017 Transformer, each a few lines of code (Exercises 1-4); checking each against a reference catches bugs that would otherwise only show up as a slightly worse loss curve.
- At GPT-2 shapes, the feed-forward network dominates the compute, until the T^2 attention term takes over at long contexts (Exercise 4).
- A 0.8M-parameter model trained for minutes on a CPU already picks up words and phrases (Exercise 5); how its text looks depends as much on decoding as on the model.
- Without RoPE, this model loses about 0.7 nats per byte on all three tested seeds (Exercise 6) - for one model size, budget and tokenizer.